

安全地写下一个字符串

`.code`
`ssize_t sio_puts(char s[]) { # ssize_t 是 POSIX 的一个符号常量, 用于字节数或源代码
| return write(STDOUT_FILENO, s, sio_strlen(s)); }`

安全地写下一个双精度数(十进制)

`.code`
`size_t sio_putl(long l) {
| char s[128];
| sio_ltoa(l, s, 10);
| return sio_puts(s); }`

安全地写出错码信息并退出程序

`.code`
`void sio_err(char s[]) {
| sio_puts(s);
| _exit(1); } # 不用 exit 是因为 exit 会调用 atexit 注册的清理函数并冲淡 stdio 缓冲区, 不要主`

7. 对于包装函数, 只写一个函数, 样子都相同

`.code`
`ssize_t sio_puts(char s[]) {
| ssize_t n;
| if ((n = sio_puts(s)) < 0) ...
| | sio_error("sio_puts error");
| return n; }`

⇒ 可以完全避免死锁和数据损坏。

其中的封装函数用 `static key word`, 其作用域被严格限制于当前。C 语言, 对外部作用着完全不可见, 且可以防止命名冲突。